

Excepciones en Java

¿Qué es una excepción?

Una excepción es un evento que interrumpe el flujo normal de ejecución de un programa. En Java, las excepciones son objetos que **se generan cuando ocurre un error en tiempo de ejecución**.

Ejemplos de situaciones que pueden provocar excepciones:

- Intentar acceder a un índice fuera de los límites de un array (**ArrayIndexOutOfBoundsException**)

```
int a[]=new int[5];  
a[10]=50; //ArrayIndexOutOfBoundsException
```

- Dividir un número por cero (**ArithmeticException**)

```
int a=50/0; //ArithmeticException
```

- Intentar acceder a un objeto que es `null` (**NullPointerException**)

```
String s=null;  
System.out.println(s.length()); //NullPointerException
```

- Convertir a numérico un valor que tiene caracteres alfabéticos

```
String s="abc";  
int i=Integer.parseInt(s); //NumberFormatException
```

- Fallos en operaciones de entrada/salida (archivos, conexiones, etc.). (**FileNotFoundException, IOException, ...**)

```
File archivo = new File("archivo_inexistente.txt");  
Scanner lector = new Scanner(archivo); // Lanza  
FileNotFoundException si el archivo no existe
```

¿Qué ocurre cuando se produce una excepción?

Supongamos que tenemos 10 líneas de código y que al ejecutarse la instrucción 5 se genera una excepción (un error). Si se da esta situación, las líneas de código que hay por detrás no se llegarán a procesar porque el programa se detendrá.

1. statement 1;
2. statement 2;
3. statement 3;
4. statement 4;
5. statement 5; *//exception occurs*
6. statement 6;
7. statement 7;
8. statement 8;
9. statement 9;
10. statement 10;

Lo único que veríamos es el detalle de la excepción que se ha producido, y toda la secuencia de llamadas que se produce desde el main hasta la instrucción que genera el error (**stack trace o pila de llamadas**). Cuando ocurre esto decimos que se ha producido un error no controlado y el programa termina abruptamente.

Debemos evitar los errores incontrolados. Debemos tratarlos para dar un mensaje personalizado al usuario informando de la situación, o intentar solucionarlo de alguna forma.

¿Cómo gestionar una excepción?

Las excepciones se pueden producir en cualquier punto del código que se está ejecutando (main, función, subfunción, dentro del mismo package, en otro package). En el momento en que se produce debemos decidir el tratamiento a realizar:

1. Tratar el error (manejarlo)
2. Propagar el error (comunicarlo a una capa o nivel superior)

Existen 5 palabras clave para el manejo de excepciones en java:

Palabra clave	Descripción
try	La palabra clave "try" se utiliza para especificar un bloque donde se debe colocar el código que puede lanzar excepciones. El bloque <code>try</code> debe ir seguido de un bloque <code>catch</code> o <code>finally</code> . No se puede usar <code>try</code> solo.
catch	El bloque <code>catch</code> se utiliza para manejar excepciones. Debe estar precedido de un bloque <code>try</code> , lo que significa que no se puede usar <code>catch</code> solo. Puede ser seguido opcionalmente por un bloque <code>finally</code> .
finally	El bloque <code>finally</code> se utiliza para ejecutar código importante del programa. Este bloque se ejecuta independientemente de si la excepción fue manejada o no.
throw	La palabra clave <code>throw</code> se utiliza para lanzar una excepción explícitamente.
throws	La palabra clave <code>throws</code> se utiliza para declarar excepciones en la firma de un método. No lanza una excepción, simplemente indica que el método podría lanzar una excepción.

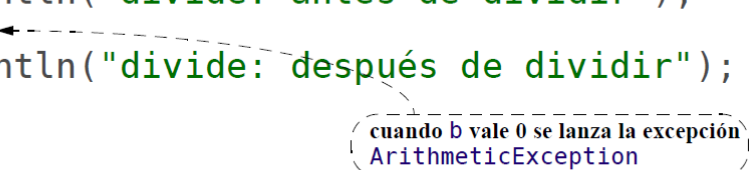
Manejar la excepción (try - catch)

```
try {  
    instrucciones;  
} catch (ClaseExcepción1 e) {  
    instrucciones de tratamiento;  
} catch (ClaseExcepción2 | ClaseExcepción3 e) {  
    instrucciones de tratamiento;  
}
```

- Los "catch" se evalúan por orden: una excepción se atrapa en el primer "catch" específico para esa excepción o para una de sus superclases

Ejemplo: propagación de excepciones

```
private static int divide(int a, int b) {
    System.out.println("divide: antes de dividir");
    int div = a/b;
    System.out.println("divide: después de dividir");
    return div;
}
private static void intermedio() {
    System.out.println("intermedio: antes de divide");
    int div = divide(2,0);
    System.out.println("intermedio: resultado:" +div);
}
public static void main(String[] args) {
    System.out.println("main: antes de intermedio");
    intermedio();
    System.out.println("main: después de intermedio");
}
```



A dashed arrow points from the `int div = a/b;` line in the `divide` method to a callout box. The callout box contains the text: "cuando b vale 0 se lanza la excepción `ArithmeticException`".

La salida generada será:

```
main: antes de intermedio
intermedio: antes de divide
divide: antes de dividir
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Propaga.divide(Propaga.java:14)
    at Propaga.intermedio(Propaga.java:21)
    at Propaga.main(Propaga.java:27)
```

En el ejemplo "propagación de excepciones" anterior, añadimos un bloque `try-catch` al método `intermedio`:

```
private static void intermedio() {
    try {
        System.out.println("intermedio: antes de " +
                           "divide");
        int div=divide(2,0);
        System.out.println("intermedio: resultado:" +
                           div);
    } catch (ArithmeticException e) {
        System.out.println("intermedio: cazada " + e);
    }
}
```

La salida por consola que obtenemos ahora es:

```
main: antes de intermedio
intermedio: antes de divide
divide: antes de dividir
intermedio: cazada ArithmeticException: / by zero
main: después de intermedio
```

- en este caso la **excepción es cazada**, por lo que
 - el **programa NO finaliza** de forma abrupta
 - NO aparece un mensaje del sistema indicando que se ha producido una excepción

En el ejemplo anterior, el manejador realiza **únicamente** el tratamiento de la excepción `ArithmeticException`

```
try {  
    ...;  
} catch (ArithmeticException e) {  
    ...;  
}
```

Es posible poner un **tratamiento común** para cualquier excepción

```
try {  
    ...;  
} catch (Exception e) {  
    ...;  
}
```

- es cómodo pero **no es recomendable**, ya que puede ocurrir un tratamiento inadecuado para una excepción no prevista

Propagar la excepción (Throws Exception)

```
// Método que puede lanzar una excepción  
private static int divide(int a, int b) throws ArithmeticException  
{  
    System.out.println("divide: antes de dividir");  
    int div = a / b; // Aquí se lanza la excepción si b == 0  
    System.out.println("divide: después de dividir");  
    return div;  
}  
  
// Método intermedio que no maneja la excepción, solo la propaga  
private static void intermedio() throws ArithmeticException {  
    System.out.println("intermedio: antes de divide");  
    int div = divide(2, 0); // Se propagará la excepción si ocurre  
    System.out.println("intermedio: resultado: " + div);  
}  
  
// Método main, que ahora debe capturar la excepción  
public static void main(String[] args) {  
    System.out.println("main: antes de intermedio");  
    try {  
        intermedio(); // Si la excepción ocurre, se propagará hasta  
aquí  
    } catch (ArithmeticException e) {  
        System.out.println("main: Excepción capturada -> " + e);  
    }  
    System.out.println("main: después de intermedio");  
}
```

La salida esperada en consola es:

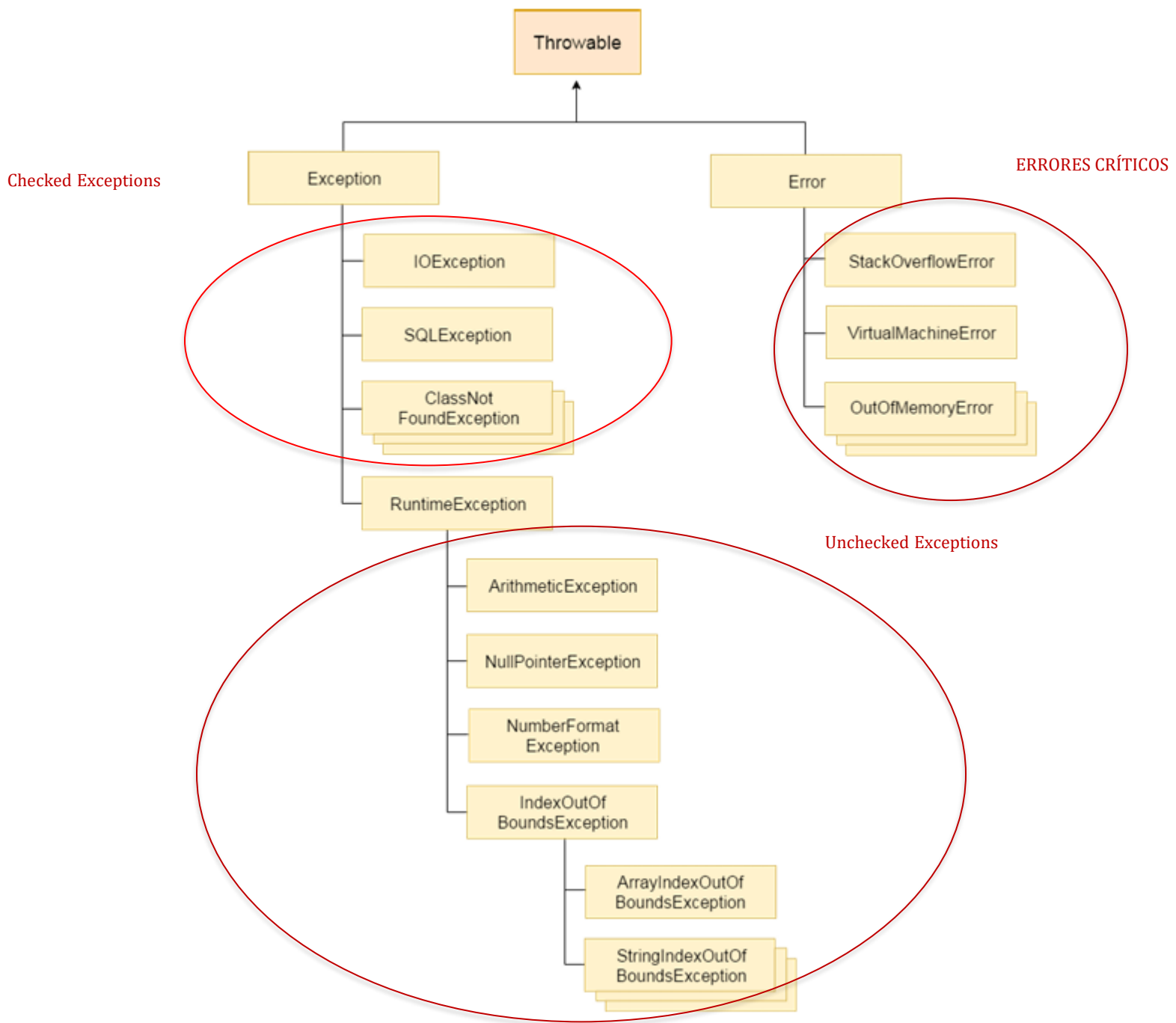
```
main: antes de intermedio
intermedio: antes de divide
divide: antes de dividir
main: Excepción capturada -> java.lang.ArithmeticException: / by zero
main: después de intermedio
```

- Si una excepción no se captura en un método, se propaga al método que lo invocó, y así sucesivamente, hasta llegar al método `main`. Si no se captura en `main`, el programa termina con un error.

Jerarquía de clases de excepciones

La clase base para todas las excepciones en Java es **Throwable**. Esta se divide en:

- **Exception:** Para errores que se pueden manejar en el programa.
- **Error:** Para problemas críticos que normalmente no se pueden solucionar.



Tipos de excepciones

En Java, las excepciones se dividen en tres tipos principales:

- **Checked Exceptions** (Excepciones verificadas):
 - **Heredan de la clase `Exception`**
 - El compilador **obliga a tratarlas mediante `try/catch` o declararlas con `throws`**
 - **Ejemplo:** `IOException`, `SQLException`
- **Unchecked Exceptions** (Excepciones no verificadas):
 - **Heredan de `RuntimeException`**
 - **No es obligatorio tratarlas** en tiempo de compilación.
 - **Ejemplo:** `NullPointerException`, `ArithmeticException`
- **Error:**
 - **Heredan de la clase `Error`**
 - **Son problemas graves del sistema que no se pueden manejar** (como un fallo de memoria)
 - **Ejemplo:** `OutOfMemoryError`, `StackOverflowError`

Creación de excepciones personalizadas

Es posible definir nuestras propias clases de excepciones extendiendo “`Exception`” o “`RuntimeException`”.

Ejemplo:

```
public class Division {
    private int numerador;
    private int denominador;

    public Division(int numerador, int denominador) throws IndeterminacionEx {

        if (denominador == 0){
            throw new IndeterminacionEx("Division entre 0");
        }

        this.numerador = numerador;
        this.denominador = denominador;
    }

    ...
}
```

```
public class IndeterminacionEx extends Exception{
    public IndeterminacionEx(String errorMess){
        super(errorMess);
    }
}
```

```
public static void main(String[] args) {
    try {
        Division division = new Division(10, 0);
        division.mostrarDivision();
    } catch (IndeterminacionEx ex) {
        System.out.println(ex.getMessage());
        //ex.printStackTrace();
    }
    System.out.println("El programa continúa ...");
}
```

Reglas de sobreescritura con excepciones

Cuando un método en una subclase sobrescribe un método de la clase padre:

- Puede lanzar las mismas excepciones declaradas en el método padre.
- Puede lanzar subclases de esas excepciones.
- No puede lanzar nuevas excepciones que no estén declaradas en el método padre.

Ventajas del manejo de excepciones

- Permite mantener el flujo normal del programa.
- Facilita el rastreo y la depuración de errores.
- Mejora la robustez y la confiabilidad de las aplicaciones.